

Lucrare practică nr. 2

Tema: Crearea și aplicarea unei rețele neuronale convoluționale (CNN)

Scopul lucrării: Implementarea unei rețele neuronale convoluționale (CNN) pentru clasificarea imaginilor în Python

Obiectivele lucrării :

1. Crearea unei rețele neuronale convoluționale (CNN) pentru clasificarea imaginilor.
2. Antrenarea rețelei și validarea rezultatelor.
3. Compararea rezultatelor cu alte rețele precum Inception, ResNet ș.a.

Numărul de ore necesare pentru realizare – 4 ore academice.

6.1 Noțiuni generale :

CNN este un algoritm de învățare profundă care poate prelua la intrarea o imagine și atribuie ponderi și părtiniri diferitor obiecte din imagine pentru a le diferenția una de cealaltă [18].

A avea un set mare de date este crucial pentru performanța modelului de învățare profundă. Cu toate acestea, poate lipsi cantitatea și diversitatea datelor. Augmentarea datelor este o strategie care permite practicienilor să crească semnificativ diversitatea datelor disponibile pentru modelele de antrenament, fără a colecta efectiv date noi.

```
# Data Augmentation
data_augmentation = tf.keras.Sequential([
    tf.keras.layers.experimental.preprocessing.RandomFlip("horizontal"),
    tf.keras.layers.experimental.preprocessing.RandomRotation(0.2),
    tf.keras.layers.experimental.preprocessing.RandomZoom(height_factor=(-
        0.3, -0.2))
])
```

E important de atras atenția la dimensiunea imaginilor! Să presupunem că avem un set de date cu dimensiunea imaginii (1000,1000), adică 1000 de pixeli fiecare pe verticală și orizontală. Aplatizarea imaginii (1000,1000) are ca rezultat o dimensiune a vectorului de 1 milion, care este inimaginabil de mare și poate exploda dimensiunea rețelelor neuronale.

Elementele de bază ale rețelelor de convoluție sunt:

1. Stratul **convoluțional** – aplicarea filtrelor de extragere a caracteristicilor (revizuiți materialul despre filtre din îndrumarul Tehnici de programare în Python [19]). Este posibil ca filtrul să nu detecteze caracteristicile importante prezente în colțurile imaginii. **Padding** ne ajută să rezolvăm această problemă, se adaugă un cadru suplimentar de zerouri la imagine pentru a permite mai mult spațiu de acoperit pentru nucleu. Adăugarea de umplutură (padding) la o imagine procesată de un RNC permite o analiză mai precisă a imaginilor. Așa cum operațiunea de convoluție scade dimensiunea hărții caracteristicilor de ieșire în comparație cu intrarea, umplutura aici ajută la mărirea hărții caracteristicilor de ieșire.

2. **Pooling** (preia harta caracteristicilor ca intrare și reduce dimensiunea selectând Max sau Average peste dimensiunea acesteia – se aplică în general pe o regiune 2x2). Folosind gruparea, se selectează cea mai bună caracteristică și se transmite înainte, lăsând în urmă restul semnalelor din regiunea înconjurătoare.

3. Straturi finale. Se adaugă câteva **straturi Dense** care sunt folosite pentru a prezice rezultatul și se selectează funcția de activare softmax (clasificare multclasă) sau sigmoide (clasificare binară) în stratul final.

Să analizăm un exemplu de creare a unei rețele neuronale convoluționale pentru clasificarea imaginilor [20]. Având deja experiență în citirea fișierelor, importarea librăriilor, pregătirii datelor, scrierii funcțiilor în limbajul Python în continuare vor fi doar enumerați acești pași fără detalieri.

Pasul 1. Importați bibliotecile necesare și vizualizați informația despre mapa cu imagini.

Pasul 2. Citiți primele 10 imagini din fișier.

Pasul 3. Amestecați imaginile în fișier cu scopul evitării antrenării rețelei doar pe un anumit tip de imagini.

Pasul 4. Decodați imaginile.

Pasul 5. Obțineți etichetele și transformați-le în valori numerice : 0 și 1 (pentru clasificarea binară)

Pasul 6. Creați setul de antrenament.

Pasul 7. Vizualizați câte un exemplu din fiecare clasă (din setul de antrenament) pentru a ne asigura că este citit corect.

Pasul 8. Creați modelul rețelei neuronale convoluționale:

```
# Example of Convolutional Neural Network (CNN)
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Activation, Conv2D,
    MaxPooling2D, Flatten, Dropout, BatchNormalization ,
    GlobalMaxPool2D
model = Sequential()

# Block 1
model.add(Conv2D(input_shape=(224 , 224 , 3),
    padding='same',filters=32, kernel_size=(7, 7)))
model.add(Activation('relu'))
model.add(BatchNormalization())
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Dropout(0.2))

# Block 2
model.add(Conv2D(filters=64, padding='valid', kernel_size=(5, 5)))
model.add(Activation('relu'))
model.add(BatchNormalization())
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Dropout(0.2))

# Block 3
model.add(Conv2D(filters=128, padding='valid', kernel_size=(3, 3)))
model.add(Activation('relu'))
model.add(BatchNormalization())
model.add(MaxPooling2D())
model.add(Dropout(0.2))

# Block 4
model.add(Conv2D(filters=256, padding='valid', kernel_size=(3, 3)))
model.add(Activation('relu'))
```

```

model.add(BatchNormalization())

model.add(Conv2D(filters=256 , kernel_size=(3, 3)))
model.add(Activation('relu'))
model.add(BatchNormalization())
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Dropout(0.2))

model.add(GlobalMaxPool2D())

model.add(Dense(units=256))
model.add(Activation('relu'))
model.add(Dropout(0.2))

model.add(Dense(units=1))
model.add(Activation('sigmoid'))

```

Pasul 9. Vizualizați informații despre modelul creat cu funcția `model.summary()`

Pasul 10. Compilați și antrenați modelul.

Pasul 11. Evaluați modelul.

1.2 Sarcina individuală:

1. Consultați <https://www.kaggle.com/datasets> pentru a alege și a descărca un set de date pentru clasificare și anume să conțină imagini.
2. Parcurgeți pașii 1-11 ca în exemplu de mai sus.
3. Comparați rezultatele obținute testând rețele precum ResNet, Inception ș.a.
4. Scrieți concluzii.

1.3 Evaluare și livrabile

Studentii vor demonstra cunoștințele și abilitățile dobândite ca urmare a realizării lucrării practice. Vor explica pașii de creare a unei rețele neuronale convoluționale, ce influențează alegerea numărului de neuroni, numărului de straturi, vor răspunde la întrebări referitoare la funcții de activare, dropout, batch normalisation, pooling, padding etc.

Studentii vor prezenta un fișier cu extensia .py și un raport .doc care conține: tema, obiectivele, sarcina, codul sursă și capturi de ecran cu rezultatele obținute la rularea codului și o concluzie despre ce au reușit să asimileze efectuând sarcinile din lucrare.

Modelul de raport poate fi consultat în Anexa 1.

Întrebări de autoevaluare:

1. Care sunt pașii de creare a unei rețele neuronale convoluționale (CNN)?
2. Ce reprezintă un strat convoluțional? Cum alegem numărul de filtre, mărimea lor? Ce este padding?

3. Cum alegem numărul de straturi convoluționale? Dar numărul de straturi Dense într-o rețea CNN?
4. Care este diferența între max-pooling și global max-pooling? Cunoașteți alt tip de straturi de pooling?
5. Ce funcții de activare folosim în straturile interne? dar în stratul de ieșire?
6. Ce calculează funcția loss?
7. Cum ne ajută optimizatorii?
8. Când și cu ce scop folosim straturile de abandon (Dropout Layers)? Ce rată de abandon e recomandată?
9. Care e scopul normalizării loturilor (Batch Normalization)?
10. Care sunt pașii de optimizare a unei CNN?